POKHARA UNIVERSITY

APEX COLLEGE
Department of Management

PROJECT REPORT
ON

PriceJach
"Image Based Price Comparison System"

BY
Aayush Neupane(23100086)

Jiban Khatri(23100101)

KATHMANDU, NEPAL

2026

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our class teacher **,** Mr. Sajjan Acharya, for his continuous guidance, valuable suggestions, and encouragement throughout the development of our project PriceJach. His support played a key role in helping us stay focused and complete this project on time.

We are also thankful to Apex College for providing us with the academic environment and resources needed for this project. A special thanks to our classmates who helped us in giving insights at the time of need. Their support and belief in our idea helped us shape PriceJach into a practical and useful system.

Abstract

PriceJach Nepal is an image-based price comparison system built using Flask for backend, and HTML with Tailwind CSS for the frontend. This system helps users find the best price of a product across multiple online sellers in Nepal. The user uploads a photo of a product, and the system automatically reads the text from the image, identifies the product, and compares its price across four sellers that are SastoDeal, Daraz Nepal, Amazon, and HamroBazar.

The system uses Tesseract OCR to extract text from the uploaded product image. Before OCR, the image goes through preprocessing steps like grayscale conversion, resizing, and Otsu's thresholding to improve text readability. The extracted text is then passed to a Multinomial Naive Bayes classifier trained on TF-IDF features to predict the product category. A hybrid approach is used where known brand keywords are checked first, and the ML model is used as a fallback. After category prediction, FuzzyWuzzy string matching is used to find the closest matching product from our dataset of 237 unique products across 5 categories that is Personal Care, Food & Beverages, Cosmetics, Household, and Sports. The system then pulls prices from all four sellers and shows the comparison along with how much the user can save.

The dataset contains around 58,000 price records collected from Kaggle and manually created data. The ML model achieved 75% accuracy on test data(36 correct out of 48 test samples), and the full pipeline with keyword-based category detection achieved 90% accuracy on manual testing(9 correct out of 10 manual test cases). All prices are shown in NPR (Nepalese Rupees).

Table of Contents

List of Tables

List of Figures

CHAPTER I

INTRODUCTION

1.1 Background

Online shopping in Nepal has grown a lot in the past few years. Platforms like Daraz, SastoDeal, HamroBazar are now commonly used by Nepalese buyers. But when people want to buy a product, they have to open each platform separately, search the product manually, and compare prices one by one. This takes a lot of time and effort. Sometimes people end up paying more just because they did not check all the platforms. After noticing this common problem, we decided to find a better way. We found that Nepal does not have a system where users can simply upload a product photo and get price comparison across multiple sellers automatically. We chose a simple and effective idea which is an Image-Based Price Comparison System called PriceJach . This system lets users upload a product image, the system reads the product text from the image using OCR, identifies the product using ML classification, and compares its price across SastoDeal, Daraz Nepal, Amazon, and HamroBazar. We have designed our system to be simple, fast, and made specially for Nepalese users with all prices shown in NPR.

1.2 Scope

Designing, creating, and implementing a web-based system that automates product identification from images and compares prices across multiple e-commerce platforms is part of the PriceJach Nepal project's scope. The system is designed for Nepalese online shoppers, enabling them to:

- Upload a product image for identification.
- Get the product name, category, and brand detected automatically using OCR and ML.
- Compare prices of the same product across four sellers — SastoDeal, Daraz Nepal, Amazon, and HamroBazar.
- See how much they can save by choosing the cheapest seller.
- The system currently covers 5 product categories : Personal Care, Food & Beverages, Cosmetics, Household, and Sports where 237 unique products are in the dataset.

1.3 Project Description

PriceJach Nepal is a web-based image-based price comparison system that helps users find the best price of a product by simply uploading its photo. The system processes the image, extracts text, identifies the product, and shows prices from multiple sellers side by side.

The primary goal of PriceJach Nepal is to save time and money for users who shop online by removing the need to manually search and compare prices on different

platforms. By using OCR and ML together, the system automates the entire process from image to price comparison.

Key Features of PriceJach Nepal:

1. **Image Upload with Drag and Drop:** Users can upload a product image by dragging and dropping it or by clicking to browse. The system supports JPG and PNG formats and shows a preview of the selected image before processing.
2. **Image Preprocessing:** The uploaded image goes through preprocessing steps — grayscale conversion, 2x resizing, and Otsu's thresholding — to make the text in the image clearer and easier to read for OCR.
3. **OCR Text Extraction:** The system uses Tesseract OCR to read text from the processed image. It also calculates OCR confidence and rejects images where the text quality is too low (confidence below 25 or fewer than 2 readable words).
4. **Brand Detection:** A brand hint dictionary handles common OCR misspellings (e.g., "lifebouy" is mapped to "lifebuoy") so the system can still identify the product even when OCR misreads the brand name.
5. **ML Category Classification (Hybrid Approach):** The system uses a 3-level approach to predict the product category — first it checks a keyword dictionary for known brands, then checks for partial brand matches, and finally falls back to a trained Multinomial Naive Bayes model with TF-IDF vectorization. The ML model was trained on 237 unique products across 5 categories.
6. **Fuzzy Product Matching:** After category prediction, the system uses FuzzyWuzzy string matching to find the closest product in the dataset. It combines product name matching (80% weight) and brand matching (20% weight) to calculate a match score out of 100.
7. **Price Comparison:** The system pulls prices from all four sellers for the matched product, keeps the lowest price per seller, sorts them, and shows the comparison. It also calculates and displays how much the user can save by choosing the cheapest option.
8. **Error Handling:** If the system cannot identify the product (low OCR quality, low match score, or low ML confidence), it shows a clear error message along with the OCR extracted text so the user knows what went wrong.

1.4 Objectives

- To help online shoppers in Nepal compare product prices across multiple e-commerce platforms quickly and easily.
- To build a system that can automatically identify a product from its image using OCR and ML classification.
- To compare prices of the same product across SastoDeal, Daraz Nepal, Amazon, and HamroBazar and show the best deal.
- To reduce the time and effort users spend manually searching and comparing prices on different websites.
- To provide a simple and easy-to-use web interface.

- To understand the basics of OCR and category classification.

CHAPTER II

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the existing research and technologies related to the key components used in our system like Optical Character Recognition (OCR), image preprocessing, text classification using machine learning, fuzzy string matching, and price comparison approaches. The purpose of this review is to understand what methods already exist and how they connect to the techniques we have used in PriceJach .

2.2 Optical Character Recognition (OCR)

OCR is the process of converting text inside an image into machine-readable text. It has been used for decades in document scanning, license plate reading, and now in product identification. Among the available OCR engines, Tesseract OCR is one of the most widely used open-source tools. Originally developed by Hewlett-Packard in the 1980s, it was later released as open-source and is now maintained by Google. Tesseract supports over 100 languages and uses an LSTM-based neural network engine (OEM 3) for better accuracy on complex text.

However, raw product images are often noisy, blurry, or have uneven lighting. This is why preprocessing the image before passing it to OCR is important, which leads to the next section.

2.3 Image Preprocessing for OCR

Research has consistently shown that OCR accuracy improves significantly when the input image is preprocessed. Common preprocessing steps include grayscale conversion, resizing, noise removal, and thresholding.

Grayscale conversion reduces the image from 3 color channels (RGB) to a single channel, which simplifies processing. Resizing the image to a larger size helps OCR read smaller text more accurately. Thresholding converts the image into pure black and white, making the text stand out clearly from the background.

In our system, we apply exactly these steps using OpenCV — converting to grayscale, resizing by 2x using cubic interpolation, and applying Otsu's threshold. We also check if the image is mostly dark after thresholding and invert it if needed, so that the text is always dark on a light background. These steps directly follow what the literature recommends for improving OCR on product labels.

2.4 Text Classification Using Naive Bayes

Text classification is the task of assigning a category or label to a piece of text. In our case, the OCR-extracted text from a product image needs to be classified into one of 5 categories which are Personal Care, Food & Beverages, Cosmetics, Household, or Sports.

Multinomial Naive Bayes (MNB) is one of the most commonly used algorithms for text classification. It is based on Bayes' theorem and assumes that words in a text are independent of each other (the "naive" assumption). Despite this simplification, it works surprisingly well for text data. McCallum and Nigam (1998) compared different variants of Naive Bayes for text classification and found that the Multinomial model performs better than the Bernoulli model because it considers word frequency, not just word presence.

2.5 Keyword-Based and Hybrid Classification

Pure ML models can sometimes fail when the input text is short or noisy, which is common with OCR output. To handle this, many systems use a hybrid approach that combines rule-based methods with ML.

Keyword-based classification uses a dictionary of known words or brand names mapped to categories. For example, if the text contains "shampoo" or "dove", it can be directly assigned to Personal Care without needing the ML model. This approach is fast and reliable for known products but cannot handle new or unknown products.

The hybrid approach combines both - check keywords first, and use ML as a fallback. This is exactly what our system does. The predict_category() function in our code first checks a BRAND_CATEGORY dictionary for exact keyword matches (returning 92% confidence) and partial matches (85% confidence). If no keyword is found, it falls back to the trained Naive Bayes model. This hybrid method improved our pipeline accuracy from 75% (ML alone) to 90% (hybrid pipeline), showing that combining rules with ML gives better results for this type of task.

2.6 Fuzzy String Matching

After identifying the category, the system needs to find the exact product from the dataset that matches the OCR text. Since OCR output often contains errors, misspellings, or extra characters, exact string matching does not work well. This is where fuzzy string matching comes in.

Fuzzy matching calculates a similarity score between two strings even if they are not exactly the same. The FuzzyWuzzy library, which is based on the Levenshtein distance algorithm, provides several matching methods. It is particularly useful because it ignores word order and duplicate words, focusing on the common tokens between two strings. For example, "dove shampoo 500ml daily care" and "Dove Shampoo 500ml" would still get a high match score.

2.7 Price Comparison Systems

In our system, we use a database-based approach with a CSV dataset containing prices from 4 sellers (SastoDeal, Daraz Nepal, Amazon, HamroBazar) for 237 products.

The compare_price() function groups prices by seller, keeps the lowest price per seller, sorts them, and calculates the potential savings. While this approach does not provide real-time prices, it is simpler and more reliable for an academic project and can be extended to include web scraping in the future.

CHAPTER III

DATA COLLECTION, PROCESSING, AND EDA

3.1 Introduction

Before building any ML model, it is important to first understand the data properly. In this chapter, we describe how the data was collected, how it was combined and cleaned, and what patterns and insights we found through Exploratory Data Analysis (EDA).

3.2 Data Collection

Since this is an academic project, the data was collected from two sources:

- **Kaggle datasets** — for some product categories.
- **Manually created data** — for the remaining products to cover all 5 categories.

The raw data was stored in 10 separate CSV files inside the raw folder, with 2 files per category:

CATEGORY	FILE 1	FILE 2
Personal Care	Personal_Care.csv	Personal_Care1.csv
Food & Beverages	Food_Beverages.csv	Food_Beverages1.csv
Cosmetics	Cosmetics.csv	Cosmetics1.csv
Household	Household.csv	Household1.csv
Sports	Sports.csv	Sports1.csv

Table 1 List of data files

Each file contains product information such as product name, brand, category, subcategory, seller name, and price in NPR.

3.3 Data Combining

All 10 raw CSV files were combined into a single dataset using the combine_data.py script. Each file was loaded individually using pd.read_csv() and then merged using pd.concat().

3.4 Data Cleaning

Data cleaning was done in process_data.ipynb. Three steps were performed:

Step 1 — Removing unnecessary columns:
The raw data had an in_stock column which was not needed for price comparison, so it was dropped.

Step 2 — Generating new product IDs:
The original product IDs were inconsistent, so new sequential IDs were created in the format P0001, P0002, P0003, etc.

Step 3 — Saving the cleaned data:
The final cleaned dataset was saved as products_all_fixed.csv .

3.5 Dataset Overview

After cleaning, the final dataset products_all_fixed.csv has the following structure:

Column	Description	Example
product_id	Unique ID for each record	P0001
product_name	Name of the product	Dove Shampoo 500ml
brand	Brand name	Dove
category	Product category	Personal Care
subcategory	Sub-category	Shampoo
seller	Seller/platform name	SastoDeal
price_NPR	Price in Nepalese Rupees	770.86
currency	Currency type	NPR

Table 2 Struture of products_all_fixed

Key Numbers:

- Total rows: ~58,000
- Total columns: 8
- Unique products: 237
- Categories: 5 (Personal Care, Food & Beverages, Cosmetics, Household, Sports)
- Sellers: 4 (SastoDeal, Daraz Nepal, Amazon, HamroBazar)

3.6 Exploratory Data Analysis (EDA)

EDA was performed in eda_allproducts.ipynb using pandas, matplotlib, and seaborn. The purpose of EDA was to understand the data distribution, identify patterns, and

find any issues before training the ML model. A total of 8 charts were generated and saved in the EDA charts/ folder.

3.6.1 Basic Statistics

We first looked at the basic statistics of product prices using df['price_NPR'].describe(). This gave us the count, mean, standard deviation, minimum, maximum, and quartile values for the price column. This helped us understand the overall price range in the dataset.

3.6.2 Chart 1 — Number of Unique Products per Category

Purpose: To see how many unique products each category has.

Chart Type: Bar chart

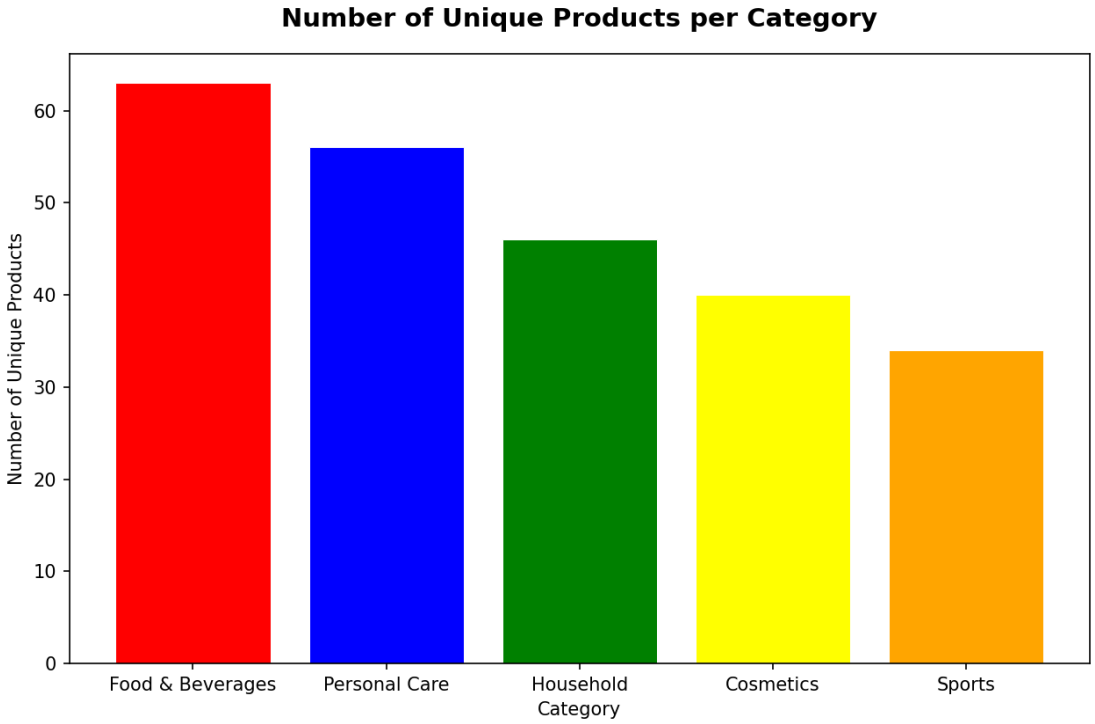

Figure 1 Number of Unique Products Per Category

Observation: This chart shows the distribution of unique products across all 5 categories. It helps us understand if the dataset is balanced or if some categories have more products than others. An imbalanced dataset can affect ML model performance, especially for categories with fewer products.

3.6.3 Chart 2 — Price Records Distribution by Category

Purpose: To see what percentage of total records each category holds.

Chart Type: Pie chart

Observation: This chart shows the share of each category in the overall dataset. Categories with more records have more price entries across sellers, giving the system more data to compare prices from. If a category has very few records, the price comparison for products in that category may be less reliable.

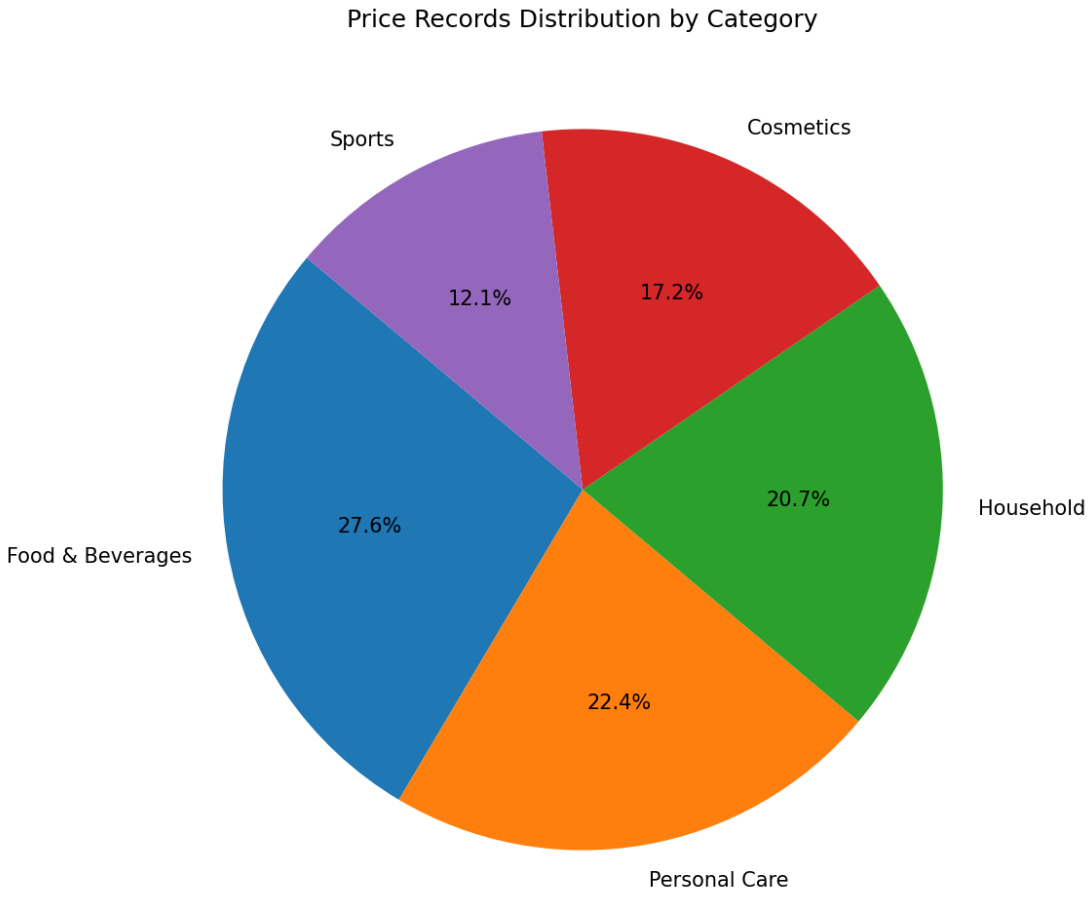

Figure 2 Price Records Distribution by Category(Pie Chart)

3.6.4 Chart 3 — Average Price per Category (NPR)

Purpose: To compare the average price across categories.

Chart Type: Horizontal bar chart

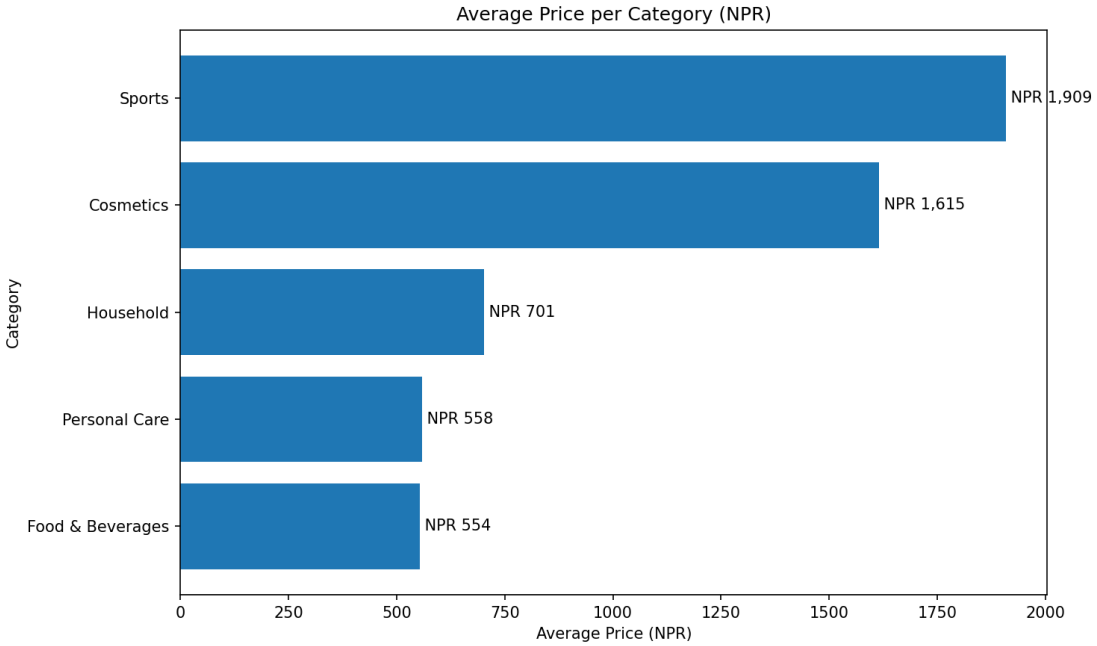

Figure 3 Average Price Price Per Category

Observation: This chart shows which category has the most expensive and cheapest products on average. It helps us understand the price range our system will deal with. For example, Sports products might be more expensive on average compared to Food & Beverages items.

3.6.5 Chart 4 — Average Price per Seller (NPR)

Purpose: To compare average prices across the 4 sellers.

Chart Type: Bar chart

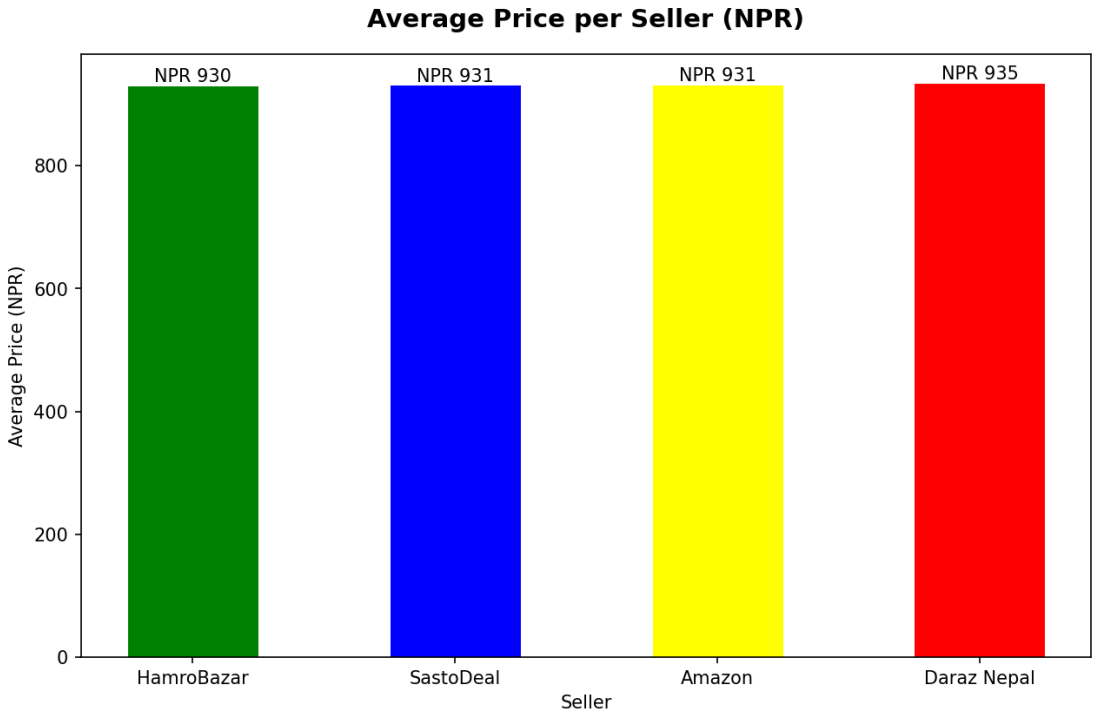

Figure 4 Average Price Per Seller

Observation: This chart shows which seller is generally cheaper or more expensive on average. It gives an idea of the pricing pattern across SastoDeal, Daraz Nepal, Amazon, and HamroBazar. However, the average alone does not tell the full story since different sellers may be cheaper for different categories.

3.6.6 Chart 5 — Price Distribution per Category (Box Plot)

Purpose: To see the price spread, median, and outliers for each category.

Chart Type: Box plot

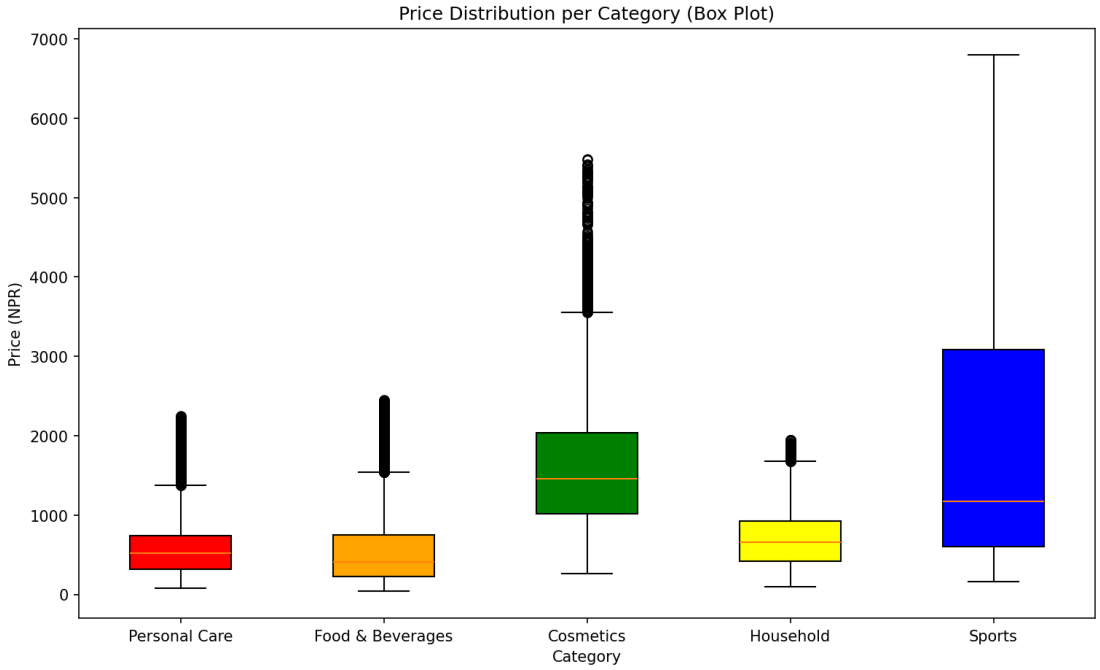

Figure 5 Price Distribution per Cateogry

Observation: The box plot shows the median price (middle line), the interquartile range (the box), and outliers (dots outside the whiskers) for each category. A wider box means more price variation within that category. Outliers indicate products that are priced much higher or lower than the norm. This helped us understand which categories have stable pricing and which have high variation.

3.6.7 Chart 6 — Average Price Heatmap (Category vs Seller)

Purpose: To see how average prices vary when we look at both category and seller together.

Chart Type: Heatmap

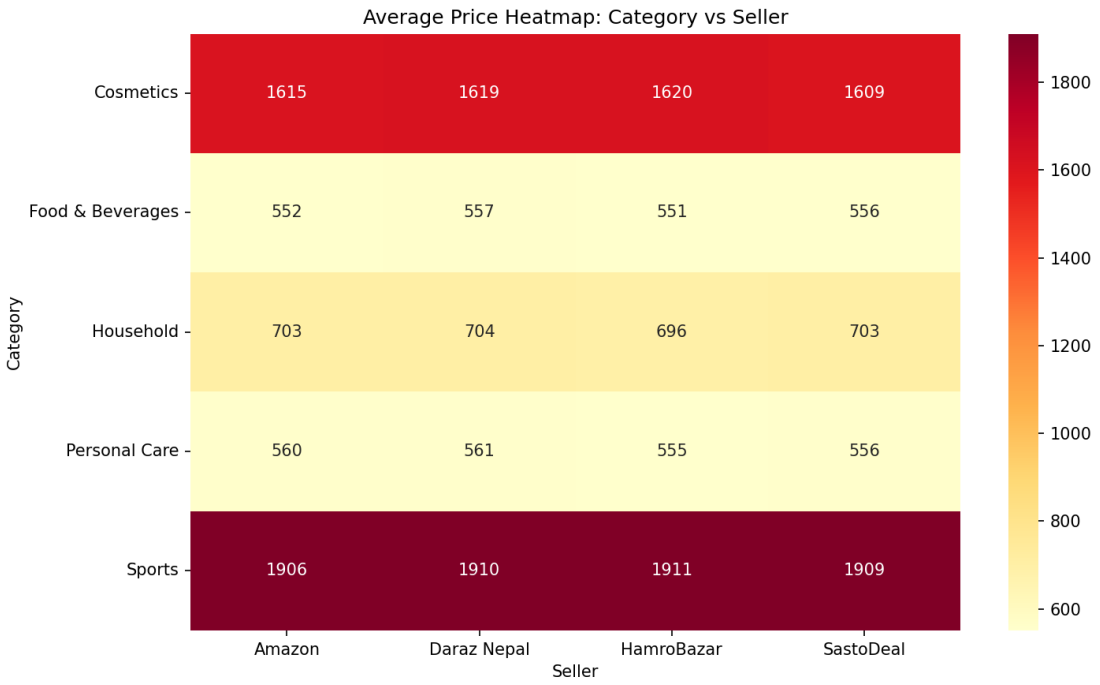

Figure 6 Average Price Heatmap: Category vs Seller

Observation: This is one of the most useful charts because it shows the average price for each combination of category and seller. For example, we can see which seller is cheapest for Personal Care products and which seller charges the most for Sports products. Darker cells indicate higher prices. This kind of analysis is exactly what our system does for individual products — but this chart shows the pattern at a category level.

3.6.8 Chart 7 — Top 10 Most Expensive Products

Purpose: To find out which products have the highest average price.

Chart Type: Horizontal bar chart

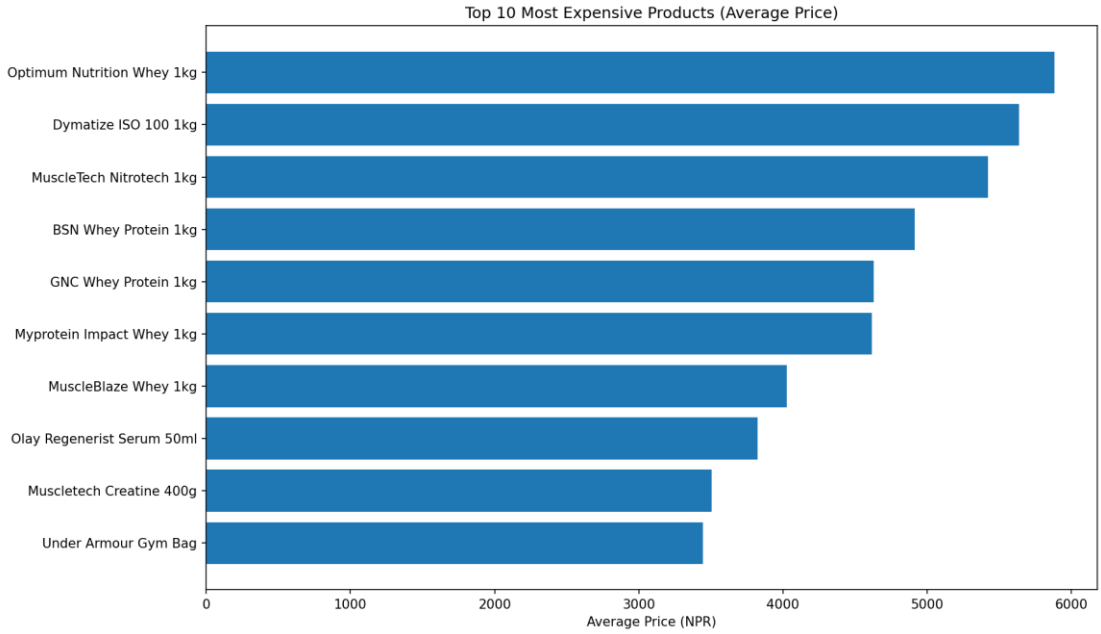

Figure 7 Top 10 most Expensive Products

Observation: This chart lists the 10 most expensive products in the dataset based on their average price across all sellers. These are usually larger-sized products or premium brands. Knowing the expensive products helps in understanding the upper price range of our system.

3.6.9 Chart 8 — Top 10 Cheapest Products

Purpose: To find out which products have the lowest average price.

Chart Type: Horizontal bar chart (red colored)

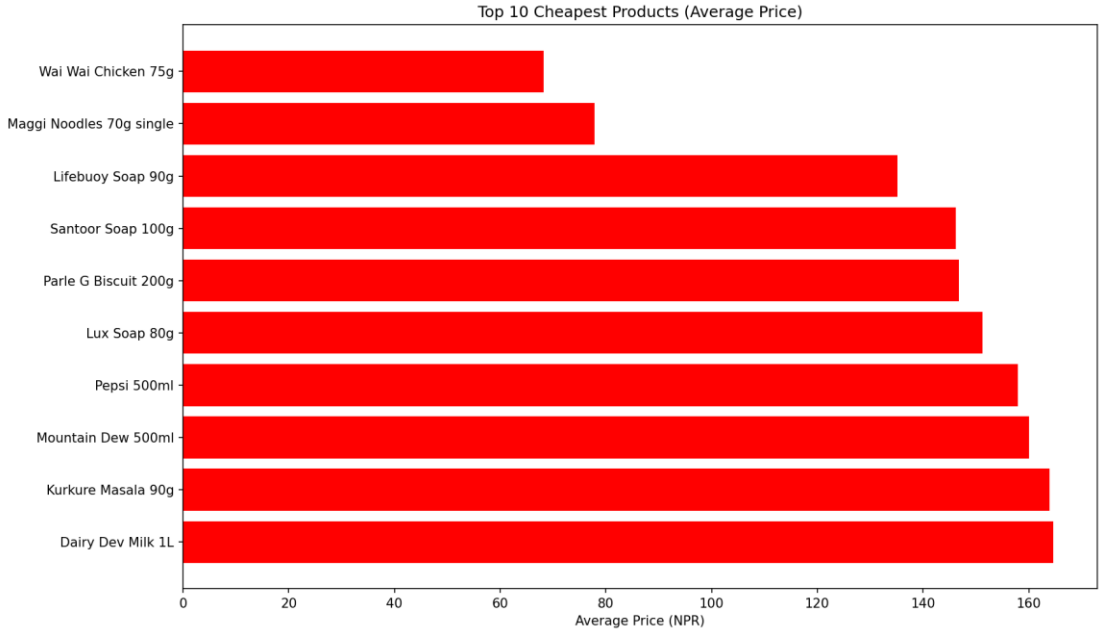

Figure 8 Top 10 Cheapest Products

Observation: This chart shows the 10 cheapest products by average price. These are usually small-sized products or basic everyday items. This helps us understand the lower end of the price range in the dataset.

3.7 Key Findings from EDA

Based on the EDA, the following key findings were noted:

1. **The dataset covers 237 unique products across 5 categories**, giving enough variety for the ML model to learn category patterns.
2. **The dataset is not perfectly balanced** — some categories like Food & Beverages and Personal Care have more records than Sports and Cosmetics. This imbalance later affected the ML model's performance on smaller categories.
3. **Prices vary across sellers for the same product**, confirming that price comparison is actually useful and there is a real difference that users can benefit from.
4. **The heatmap showed that no single seller is always the cheapest** — different sellers offer better prices for different categories, which is exactly why a comparison system is needed.
5. **Box plots revealed outliers in some categories**, showing that a few products have unusually high or low prices compared to others in the same category.
6. **All prices are in NPR and the currency column is consistent**, so no currency conversion was needed.

CHAPTER IV

MODEL SELECTION, TRAINING, AND EVALUATION

4.1 Introduction

After completing EDA and understanding the data, the next step was to build an ML model that can predict the product category from text. This chapter covers why we chose Multinomial Naive Bayes, how we prepared the data for training, how the model was trained, and how we evaluated its performance. All training was done in train_model.ipynb and evaluation was done in testing_evaluation.ipynb.

4.2 Model Selection

The main task of the machine learning model in this system is to classify a product into one of the five predefined categories: Personal Care, Food & Beverages, Cosmetics, Household, and Sports. Since the input to the model is text (product name, brand, and subcategory combined together), this is a text classification problem.

For this purpose, the **Multinomial Naive Bayes** algorithm was selected. This algorithm was chosen for the following reasons:

- It is specifically designed for text classification tasks and works well with word frequency-based features.
- It is simple, fast, and requires very little computational resources to train and predict.
- It performs well even with a small dataset, which is suitable for this project since there are only 237 unique products.
- It has been widely used in similar text classification problems such as spam detection and document categorization.

To convert the text data into numerical form that the model can understand, **TF-IDF (Term Frequency–Inverse Document Frequency) Vectorizer** was used. TF-IDF assigns a weight to each word based on how frequently it appears in a product's text and how rare it is across the entire dataset. This helps the model focus on important and distinguishing words like "shampoo," "detergent," or "protein" rather than common words that appear everywhere.

4.2 Data Preparation for Training

The dataset used for training contains 58,000 price records across 237 unique products from 4 sellers (SastoDeal, Daraz Nepal, Amazon, and HamroBazar). Each product has attributes such as product name, brand, category, subcategory, seller, and price in NPR.

For model training, a new text column was created by combining three columns: product name, brand, and subcategory. This combined text serves as the input feature

for the model. Since multiple price records exist for the same product (one per seller), duplicate product names were removed, leaving 237 unique product entries for training and testing.

The dataset was then split into two parts using an 80/20 ratio:

Split	Purpose	Number of Samples
Training Set	Model learns patterns from this data	189 (80%)
Testing Set	Model is evaluated on this unseen data	48 (20%)

Table 3 Training Set vs Testing Set Ratio

A fixed random state of 42 was used to ensure the same split is produced every time the code is run, making the results reproducible.

4.3 Model Training

The training process involved two steps:

1. **Text Vectorization:** The TF-IDF Vectorizer was fitted on the training data to build a vocabulary of all unique words and then transform each product's text into a numerical vector based on word importance.
2. **Model Fitting:** The Multinomial Naive Bayes model was trained on the vectorized training data. During training, the model learned the relationship between specific words and their corresponding product categories. For example, words like "shampoo," "soap," and "lotion" were associated with the Personal Care category, while words like "noodles," "cola," and "biscuit" were associated with Food & Beverages.

After training, both the trained model and the TF-IDF vectorizer were saved as pickle files (model.pkl and vectorizer.pkl) so they can be loaded directly by the Flask web application without retraining.

4.4 Model Evaluation

The trained model was evaluated on the 48 unseen test samples. The following results were obtained:

Overall Accuracy: 75.0% (36 out of 48 predictions were correct)

The detailed performance for each category is shown in the table below:

Category	Precision	Recall	F1 Score	Test Samples
Cosmetics	1.00	0.42	0.59	12
Food & Beverages	0.68	1.00	0.81	13
Household	0.75	1.00	0.86	3

Personal Care	0.71	1.00	0.83	12
Sports	1.00	0.38	0.55	8

Table 4 Model Evaluation for each category

Key observations from the evaluation:

- **Household** achieved the highest F1 score of 86%, followed by **Personal Care** at 83% and **Food & Beverages** at 81%. These categories had distinctive product names and keywords that the model could learn effectively.
- **Cosmetics** (59% F1) and **Sports** (55% F1) were the weakest categories. Both had high precision (1.00) but low recall (0.42 and 0.38 respectively), meaning the model was accurate when it did predict these categories but often misclassified them as other categories.
- The lower performance of Sports and Cosmetics can be attributed to fewer training samples and overlapping keywords with other categories. For example, words like "water" and "cream" appear in multiple categories.

A **confusion matrix** was generated to visualize where the model made correct and incorrect predictions. The matrix confirmed that Cosmetics products were frequently misclassified as Food & Beverages or Personal Care, and Sports products were misclassified as Food & Beverages or Personal Care.

4.5 Pipeline Testing

Beyond the ML model evaluation, a manual pipeline test was conducted to verify the complete system works end to end. Ten known products were passed through the full prediction pipeline, and the predicted categories were compared with the actual categories.

Product	**Actual Category**	**Predicted Category**	**Result**
Dove Shampoo 500ml	Personal Care	Personal Care	Correct
Colgate Toothpaste 150g	Personal Care	Personal Care	Correct
Maggi Noodles 5 pack	Food & Beverages	Food & Beverages	Correct
Ariel Detergent Powder 1.8kg	Household	Household	Correct
Pocari Sweat 500ml	Sports	Sports	Correct
Maybelline Lipstick Red	Cosmetics	Cosmetics	Correct
Nestle Milo 400g	Food & Beverages	Food & Beverages	Correct

Harpic Toilet Cleaner 1L	Household	Household	Correct
Optimum Nutrition Whey 1kg	Sports	Sports	Correct
Garnier Micellar Water 400ml	Cosmetics	Personal Care	Wrong

Table 5 Pipeline Testing

The pipeline test accuracy was **90% (9 out of 10 correct)**. The only incorrect prediction was for "Garnier Micellar Water 400ml," which was predicted as Personal Care instead of Cosmetics. This is because the word "water" is commonly associated with Personal Care products in the training data.

4.6 Price Comparison Evaluation

The price comparison feature was also tested to verify that the system correctly retrieves and compares prices across different sellers. For each test product, the system successfully identified the minimum price, maximum price, potential savings, and the best seller offering the lowest price.

4.7 Summary

Metric	**Value**
Algorithm	Multinomial Naive Bayes
Vectorizer	TF-IDF
Total Dataset Records	58,000
Unique Products	237
Categories	5
Training Samples	189
Test Samples	48
Model Accuracy	75.0%
Pipeline Test Accuracy	90.0%
Best Category (F1)	Household — 86%
Weakest Category (F1)	Sports — 55%

Table 6 Complete Evaluation Summary

The model achieves acceptable accuracy for the scope of this academic project. The keyword-based fallback system implemented in the web application further improves real-world prediction accuracy by using known brand names and product keywords to assist the ML model when OCR text is noisy or ambiguous.

CHAPTER V

Conclusion, Limitations, Future Enhancements

Conclusion

This project successfully developed an image-based price comparison system named PriceJach Nepal that allows users to upload a product image and compare its prices across four different e-commerce sellers: SastoDeal, Daraz Nepal, Amazon, and HamroBazar.

The system was built by combining multiple technologies into a single working pipeline. Tesseract OCR was used to extract text from product images, with image preprocessing techniques such as grayscale conversion, resizing, and OTSU thresholding applied to improve text extraction accuracy. The extracted text was then passed to a Multinomial Naive Bayes classifier trained with TF-IDF vectorization to predict the product category. FuzzyWuzzy string matching was used to match the OCR text with the closest product in the database. Finally, the system retrieved and compared prices from all four sellers and displayed the results to the user through a Flask web application built with HTML and Tailwind CSS.

The dataset used in this project contains 58,000 price records across 237 unique products in 5 categories: Personal Care, Food & Beverages, Cosmetics, Household, and Sports. The data was collected from Kaggle datasets and supplemented with manually created records for academic purposes. Exploratory Data Analysis (EDA) was performed to understand price distributions, category-wise product counts, and seller-wise pricing patterns before proceeding with model training.

The ML model achieved an overall accuracy of 75.0% on the test set of 48 samples. Categories such as Household (86% F1 score), Personal Care (83% F1 score), and Food & Beverages (81% F1 score) performed well due to their distinctive keywords. Cosmetics (59% F1 score) and Sports (55% F1 score) had lower performance because of overlapping keywords with other categories and comparatively fewer training samples. To address this limitation, a keyword-based fallback system was implemented in the web application, which uses known brand names and product-related keywords to assist the ML model, especially when OCR text is noisy or unclear. The end-to-end pipeline test achieved a higher accuracy of 90%, confirming that the combined system performs better than the ML model alone.

The price comparison feature was tested and verified to correctly identify the lowest and highest prices across sellers, calculate potential savings, and recommend the best seller for each product.

Limitations

- The dataset is limited to 237 unique products and 5 categories, which restricts the system to a specific set of products.
- OCR accuracy depends on the quality of the uploaded image. Blurry, low-resolution, or angled images may produce noisy text, leading to incorrect matches.
- The Cosmetics and Sports categories had weaker classification performance due to fewer training samples and overlapping keywords with other categories.
- The price data is static and does not update in real time from actual e-commerce platforms.

Future Enhancements

- The dataset can be expanded with more products and categories to improve model accuracy and coverage.
- A deep learning-based image classification model such as Convolutional Neural Network (CNN) can be used to directly classify products from images, reducing dependency on OCR.
- Real-time price scraping from e-commerce websites can be integrated to provide up-to-date price comparisons.
- The system can be deployed online so that users can access it from any device through the internet.
- Overall, this project demonstrates how OCR, machine learning, and web development can be combined to build a practical system that helps consumers find the best prices for everyday products across multiple online sellers in Nepal.